

Day - 21 - Shell Scripting Cheat Sheet

Task 1 : Basics

① Shebang (#! /bin/bash) : tells system what interpreter to use, if not written it may cause error.

② Running a script : $\rightarrow$ `chmod +x` : makes script executable.

$\rightarrow$ `./script.sh` : runs the script (if +x).

$\rightarrow$ `bash script.sh` : directly executes script if whether having +x or not.

③ Comments : $\rightarrow$ Single-Line Comment.
Eg. `# This is line.`

$\rightarrow$ Inline Comment.

Eg. `echo "HP" # This is inline.`

$\rightarrow$ Multi-Line Comment.

Eg. `<< task
Read
to use
task.`

④ Variables : $\rightarrow$ Declaring : `NAME = "Aakash"`
 $\rightarrow$ Using : `echo "$NAME"`

$\rightarrow$ Quoting : `"$VAR"` $\rightarrow$ allows expansion
`'$VAR'` $\rightarrow$ literal string.

⑤ Reading user input : Eg. `read num`

or `read -p "enter value" num.`

⑥ Command-Line Argument :

$\rightarrow$ `$0` : gives script name.

$\rightarrow$ `$1` : first Argument

$\rightarrow$ `$#` : No. of Arguments

$\rightarrow$ `$@` : All arguments

$\rightarrow$ `$?` : gives status (P/F)

of previous command.

Task 2: Operators & Conditionals.

① String Comparisons :

→ `=` : for checking strings exactly same (in any unix system).
E.g. `if ["$str1" = "$str2"]; then.`

→ `==` : same as `=` but specifically for bash.

~~→ `!=` : not equal to~~
E.g. `if ["$str1" != "$str2"]; then.`

→ `-z` : for checking if string is empty.
E.g. `if [-z "$str"]; then.`

→ `-n` : for checking if string is non-empty.
E.g. `if [-n "$str"]; then.`

② Integer Comparison :

→ `-eq` : equal E.g. `if [$n -eq 10]; then.`

→ `-ne` : not equal E.g. `if [$n -ne 10]; then.`

→ `-lt` : less than E.g. `if [$n -lt 10]; then.`

→ `-gt` : greater than E.g. `if [$n -gt 10]; then.`

→ `-le` : less or equal E.g. `if [$n -le 10]; then.`

→ `-ge` : greater or equal E.g. `if [$n -ge 10]; then.`

③ File Test Operators :

→ `-f` : true if file exists E.g. `if [-f test.txt];`

→ `-d` : true if directory exists E.g. `if [-d practice];`

→ `-e` : checks if anything exists (file, folder, path, ~~etc~~)
E.g. `if [-e "path", zip, config etc]`

→ `-r` : to check file readable.

→ -w : to check file writable.

→ -x : to check file executable

→ -s : to check file non-empty.

(4)

④ if, elif, else

Syntax :

② if [condition]; then

Command

else

Command

fi

if [condition 1]; then

Command

elif [condition 2]; then

Command

else

Command

fi

(5)

⑤ Logical Operators :

→ && : "AND" → ^{returns} true if both condition true.

Eg. if [\$n -gt 0 && \$n -lt 10]

→ || : "OR" → true if any condition true.

→ ! : "NOT" → returns false if condition true.

⑥ Case statements :

read choice

case \$choice in

start)

echo "Starting"

;;

stop)

echo "Stopping"

;;

*)

echo "Invalid"

;;

esac.

(7)

(8)

(9)

(10)

(11)

Task 3: loops.

1. for loop - list-based & C-style:

```
for i in 1 2 3
do
  echo $i
done
```

```
for ((i=1; i=5; i++))
do
  echo $i
done
```

2. while loop:

```
i=1
while [ $i -le 5 ]
do
  echo $i
  ((i++))
done
```

3. until loop:

```
i=1
until [ $i -gt 5 ]
do
  echo $i # 1 to 5
  ((i++))
done
```

4. Loop Control:

Break: stop loop

```
for i in 1 2 3 4
do
  echo "$i"
  if [ $i -eq 3 ]; then
    break
  fi
done
```

Continue: skip iteration

```
for i in 1 2 3 4
do
  if [ $i -eq 3 ]; then
    continue
  fi
  echo "$i"
done
```

5. Looping over files:

```
for file in *.log
do
  echo $file
done
```

⑥ Looping over command output:

```
while read line
do
    echo $line
done < file.txt
```

~~###~~ Task 4: Functions.

① Defining a function:

Syntax: `function-name () { ... }`

Eg. `greet () {`
 `echo "Hello"`
 `}`

② Calling a function:

`greet`

③ Passing arguments to functions:

`greet () {`
 `echo "Hello $1"`
 `}`
`greet Aakash`

Syntax:
`function-name`
`$1 $2 ...`

④ Return values:

return

(tell the script if function finished correctly)

0 → Success

(1-255) → error

Eg. `check-file () {`

`if [ -f "file.txt" ]; then`

`return 0`

`else`

`return 1`

fi

`check-file`

status = \$?

echo

(. to output data/text)

Eg. `get-name () {`

`echo "Alice"`

`}`

`name = $(get-name)`

- ⑤ Local Variables : • defined by 'local' keyword.
• range within the function.

E.g. myfunc() {

local var = 'test'

echo \$var

}

myfunc

Task 5: Text Processing Commands

① grep - Search patterns

- -i : 'Ignore Case' - performs case-insensitive search
- -v : 'Invert Match' - displays all lines that do not match specific pattern.
- -n : 'Line Number' - prefixes each matching line with its line number in file.
- -r : 'Recursive' - searches through all files in a directory & its sub-directories.
- -c : 'Count' - displays the total count of matching lines, instead of showing the lines.
- -E : 'Extended Regex' - match one pattern or other

E.g. grep -E "apple | orange" file.txt.

② awk - print columns, field separator, patterns, BEGIN/END

Syntax: awk 'pattern {action}' file

- \$1, \$2, ... : Represent the first, second etc columns.
- \$0 : represents the entire line. (row)
- NR : The current line number (No. of Records).
- NF : no. of column in current line

BEGIN : Runs once before processing the file.
END : Runs once after all lines are processed.

Eg. Print all lines of a file.

```
awk '{print}' file.txt
```

* Print specific column

```
awk '{print $2}' file.txt
```

* Print multiple columns

```
awk '{print $2, $3}' file.txt
```

* Conditional Printing.

```
awk '$2 > 80 {print $2, $3}' new.txt
```

* Pattern Matching.

```
awk '/Alice/' {print}' student.txt
```

* Average Calculation

```
awk '{sum += $1} END {print "Average :",  
sum/NR}' numbers.txt
```

* Print the numbers.

```
awk '{print NR, $0}' file.txt
```

* Filter lines by column count

```
awk 'NR == 3' file.txt
```

Field Separator : `awk -F`

E.g. Split by a single comma

```
awk -F ',' '{print $2}'
```

* Split by colon.

```
awk -F ':' '{print $2}'
```

③ sed - substitution, delete lines, in-place edit

Eg. Replace first occurrence on each line.

```
sed 's/unix /linux /' file.txt
```

→ -g : global replace

- I : Case - Insensitive

E.g. sed 's/unix/linux/gI' file.txt

→ -d: remove lines.

E.g. * Delete 3rd line.

sed '3d' file.txt

* Delete 1 to 5 lines.

sed '1, 5d' file.txt

* Delete line containing word "error".

sed '/error/d' file.txt.

→ -i: In-place editing.

E.g. sed -i 's/old/new/g' file.txt

④ cut - extract columns by delimiter (-d)

E.g. cut -d ',' -f 2 file.csv
 ↓ ↓ ↓
 delimiter file 2nd column

⑤ sort: → alphabetical = sort file.txt

→ numerical = sort -n file.txt

→ reverse = sort -r file.txt

→ unique = sort -u file.txt

→ human readable = -h

→ case - insensitive = -f

→ by column = -k E.g. sort -k 2 file.txt.

⑥ uniq: remove or count repeated lines in a file.

→ -c = count repeats E.g. sort file | uniq -c

→ default = remove duplicates E.g. sort file | uniq

→ -d = list duplicates only

⑦ tr: translate command - character-level transformations -

Eg. * lowercase to uppercase.

Eg. echo "hello" | tr 'a-z' 'A-Z'

→ -d (delete characters)

Eg. ~~tr~~ echo "Phone: 123-456" | tr -d '0-9'

⑥ wc - line/word/char count :

Eg. Count lines in a file

wc -l file.txt.

-w : word count. Eg. wc -w file.txt

-m : chars count. Eg. wc -m file.txt.

⑨ head / tail - first / last n lines, follow mode :

Eg. ~~first~~ first 5 lines.

head -n 5 file.txt.

last 5 lines.

tail -n 5 file.txt

In - follow mode.

tail -f file.txt.

Task 6 : Useful Patterns & One liners

• Find & delete files older than n days.

find /path/to/directory -type f -mtime +N -delete

If want to delete <.log> files.

find <path> -name "*.log" -mtime +N -delete.

• Count lines in all .log files.

wc -l *.log.

• Replace a string across multiple lines.

sed -i 's/old/new/g' *.txt.

- Check if service is running.

systemctl status nginx

- Monitor disk usage with alerts. alert.
of -h. → Just set threshold
& put if ~~condition~~ condition.
- Parse CSV or JSON from command line.
↓
Miller tool jq tool

Eg. * Pretty Print : `jq ' - ' file.json`.

* Extract a specific field : Access a value by its key :
`jq '.user.name' file.json`.

* Print specific columns :

`mlr --csv cut -f name, age file.csv`.

- Tail a log & filter for errors in real time

`tail -f log.txt | grep ERROR`.

Task 7: Error Handling & Debugging

① Exit Codes :

- `$?` : holds exit code of last command.
- `exit 0` : return success
- `exit 1` : return failure

② `set -e` → exit on error.

③ `set -u` → treat unset variable as error.

④ `set -o pipefail` → catch error in pipes.

⑤ `set -x` → trace execution (debug mode)

⑥ `trap` → `trap 'cleanup' EXIT` - run cleanup when script exists.
↳ function.